

Jet Classification with Graph Neural Networks

Summer School on Scientific Machine Learning, Petnica 2026

Problem statement

A jet forms when a high-energy quark or gluon, unable to exist on its own because of color confinement, showers into a cascade of particles that then hadronize into a spray of mostly pions. Each jet ends up as a variable-length, unordered set of particles, anywhere from a handful to dozens, each with its own momentum and position. Gluons carry a larger color charge than quarks, so they radiate more and produce broader, busier jets, while quark jets tend to be narrower with fewer particles carrying most of the energy.

Quantum chromodynamics, or QCD, is the theory that governs how quarks and gluons interact, and most jets produced at the LHC are ordinary byproducts of these everyday strong-force processes. Quark/gluon tagging is a routine step used to separate this common background from rarer processes physicists are actually looking for, such as the Higgs boson, discovered at the LHC in 2012, or evidence of particles beyond the Standard Model that experiments are still searching for today. Better tagging translates directly into more sensitive searches and more precise measurements.

The task is to build a binary classifier, light quark jets (up, down, strange) versus gluon jets, that predicts the correct label from each jet's raw particle content. Each particle becomes a node, connected to its nearest neighbors, and a Graph Neural Network learns from that structure directly, the same approach used by the strongest taggers in the literature, such as ParticleNet.

The data comes from the CMS Open Data Portal, record 12100, a sample of simulated 13 TeV proton proton collisions with jets reconstructed by the particle flow algorithm, including per particle kinematics and precomputed quark/gluon flavor labels. Work with a modest subset of a few thousand jets, and keep the network itself simple.

Dataset

- [Jet flavour content dataset \(record 12100\)](#): the dataset used for this project, quark and gluon jets from simulated 13 TeV collisions, with per-particle kinematics and flavor labels.

Example notebook

- [CMS Open Data QCD Jets. Machine Learning Usage Example](#): notebook showing how to load record 12100 and run a basic quark/gluon classifier.

Further exploration

- [CERN Open Data Portal](#): main portal, useful for browsing related datasets and documentation.
- [CMS Data Science / ML dataset search](#): filtered listing of other CMS datasets prepared for ML use.

Optional extension

- [Higgs→bb tagging dataset \(record 12102\)](#): a harder, boosted-jet tagging problem to try after the main task is working.

How to approach the problem?

0. Understand the problem

Before touching any data or code, make sure you understand what a jet is, why quark and gluon jets differ, and why this classification matters. Convince yourself why a GNN is the right tool for solving this problem.

1. Get to know the data

- Download a limited slice of the dataset (e.g. `JetNtuple_RunIISummer16_13TeV_MC_1.root`); don't pull the full release!
- Open it up and look around first: what fields exist, what a single jet's entry actually looks like.
- For each jet, pull out the list of particles that make it up, and record each particle's position and momentum relative to the jet. The dataset has more fields than you need, some redundant, some jet-level rather than particle-level, so decide deliberately what belongs in that feature vector. The number of particles will differ from jet to jet, that's expected.
- Check class balance and typical particle counts per jet before doing anything else.

2. Build the point-cloud pipeline

A jet's particles form a point cloud: an unordered, variable-size set of points, each with its own position and momentum, with no grid or fixed ordering tying them together. This is the natural way to represent a jet's raw content, and it's what the GNN actually operates on, but it needs to be turned into something a network can process in batches.

- Jets don't all have the same number of particles, but models usually need every input in a batch to be the same size. Pick a max particle count, pad shorter jets with empty filler particles to reach it, and keep a mask marking which particles are real so the filler ones don't affect training.
- Particle features (like momentum) can be on very different scales. Rescale them so the model trains well, and be ready to explain why you chose that particular rescaling method.

3. Set up the model

Implement a small GNN rather than a large one. The interesting part of this problem is the graph representation, not network depth.

- Input: the padded, normalized point cloud and mask from step 2, each particle represented by its feature vector.
- A graph-convolution layer that, for each particle, finds its k nearest neighbors (by distance in feature space, e.g. Euclidean distance on $dE_t/d\Phi$) and aggregates information from them, is enough. This is the core idea behind EdgeConv/ParticleNet-style models.
- Stack 2-3 of these layers, then pool over all particles in a jet (e.g. mean or max) to get a single jet-level vector, and pass that through a small classification head (one or two dense layers) to get the final prediction.
- Make sure padded particles are excluded from both steps: they should never be selectable as a neighbor, and they shouldn't contribute to the final pooled result.

4. Train

Get a trained model within a capped compute budget.

- Cap epochs deliberately. Decide the budget up front rather than training until time runs out.
- Choose a batch size, learning rate, optimizer, loss function, and train/validation/test split.
- Check whether the classes are balanced, and account for it if not (e.g. class weighting or subsampling).
- Log training and validation loss curves to check for overfitting.

5. Evaluate

Judge the model on held-out data, and try to understand its behavior, not just its accuracy.

- Report standard classification metrics (e.g. accuracy, ROC-AUC, confusion matrix) on a held-out split, and compare against the dataset's built-in quark/gluon likelihood score (`jetQGL`) as a baseline.
- If time allows, check whether particle count or jet substructure correlates with where the model gets things wrong.

If you have extra time

Given that we suppose that these jets form two distinct families, try to approach this classification problem in an unsupervised way. For example, you could try to apply K-mean clustering, or something similar. In that case you would have to define some sort of a metric, which would follow from a specific “comparison” between each two jets. Given that the jets have different number of particles, this might get a little tricky, but through the discussion among yourselves and with us, we can hopefully figure out something.